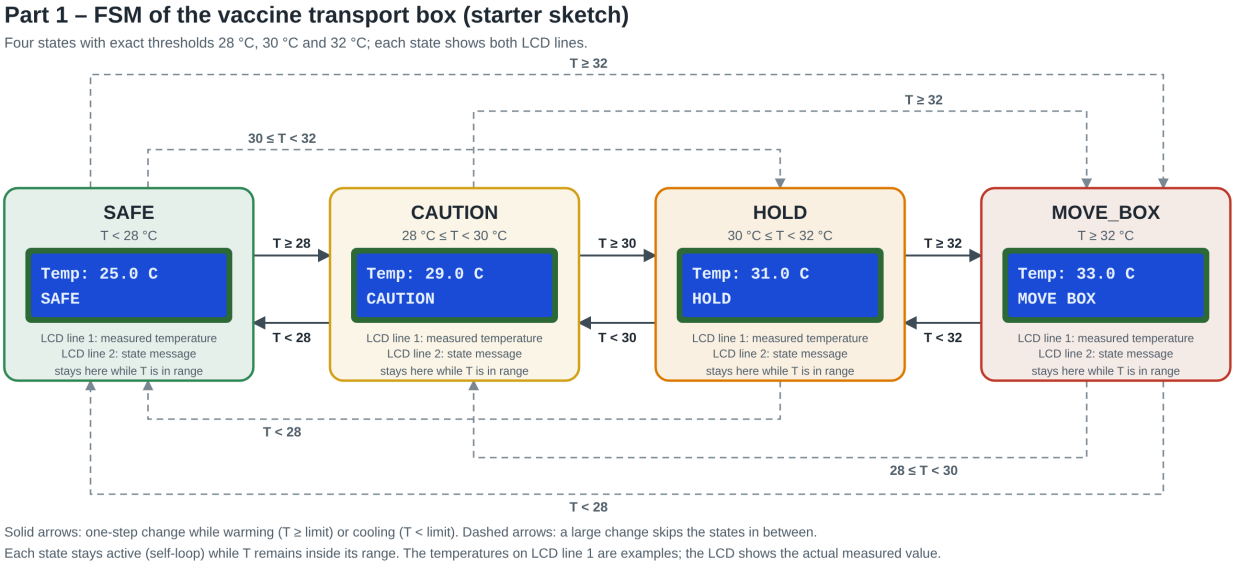
Vaccine Transport Box – Workshop Report

ECS100/ECS200 Biomedical Engineering Workshop · Oct 21, 2026 · @HAMZA

Student: Hamzeh Al Shamali · **Student ID:** 1108979 · Workshop 7 – TinkerCAD Simulation (individual submission)

Part 1 – Improved FSM

The FSM has four states, and the next state depends only on the current temperature T, so a large jump can skip intermediate states.



Part 1 FSM (starter sketch outputs). Each state shows both LCD lines: line 1 is the measured temperature value (example shown), line 2 is the state message. Solid arrows are one-step changes while warming (T ⩾ limit) or cooling (T < limit); dashed arrows show that a large change skips the states in between. The updated FSM after the Part 4 improvement is in Part 4.

State	Temperature range	LCD line 1 (example measured value)	LCD line 2
SAFE	T < 28 °C	Temp: 25.0 C	SAFE
CAUTION	28 ⩽ T < 30 °C	Temp: 29.0 C	CAUTION
HOLD	30 ⩽ T < 32 °C	Temp: 31.0 C	HOLD
MOVE_BOX	T ⩾ 32 °C	Temp: 33.0 C	MOVE BOX

Line 1 always shows the actual measured temperature with one decimal, so it changes as the sensor reading changes; line 2 shows the message of the current state.

Transitions as temperature increases

- SAFE → CAUTION when T ⩾ 28
- CAUTION → HOLD when T ⩾ 30
- HOLD → MOVE_BOX when T ⩾ 32
- Skips: SAFE → HOLD when 30 ⩽ T < 32; SAFE → MOVE_BOX or CAUTION → MOVE_BOX when T ⩾ 32

Transitions as temperature decreases

- MOVE_BOX → HOLD when T < 32
- HOLD → CAUTION when T < 30
- CAUTION → SAFE when T < 28
- Skips: MOVE_BOX → CAUTION when 28 ⩽ T < 30; MOVE_BOX → SAFE or HOLD → SAFE when T < 28

If none of these conditions is true, the FSM stays in its current state (self-loop).

Suggested change for clarity: add hysteresis of 0.5 °C on the way down (e.g. CAUTION → SAFE only when T < 27.5). Near a threshold the sensor reading jitters by about one ADC step (≈0.5 °C), so without hysteresis the LCD can flicker between two states. A second change, used in Part 4, is to replace short labels with handling instructions.

Part 2 – Base circuit

The TMP36 on A0 is the sensor input and the I2C LCD on A4/A5 is the output; both run from the Arduino 5V rail.

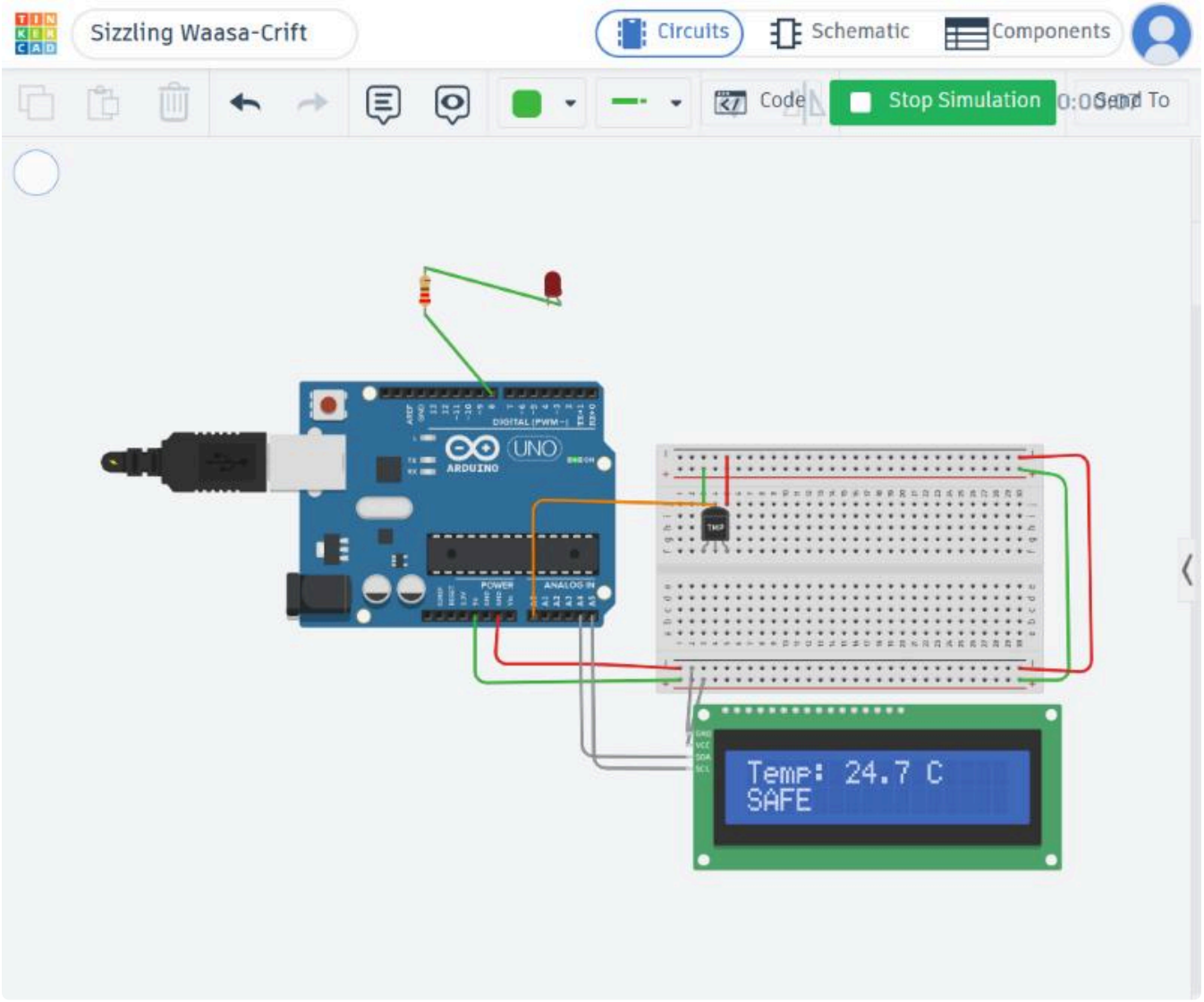
Component	Pin	Arduino Uno
TMP36	+Vs (left pin, flat side facing you)	5V
TMP36	Vout (middle pin)	A0 (sensor input)
TMP36	GND (right pin)	GND
16×2 I2C LCD	VCC	5V
16×2 I2C LCD	GND	GND
16×2 I2C LCD	SDA	A4
16×2 I2C LCD	SCL	A5
Bonus LED	Anode (long leg) via 220 Ω	Pin 8
Bonus LED	Cathode (short leg)	GND

Checks before powering: TMP36 flat side facing you (reversed it heats up), 5V and GND rails not swapped, LCD address set correctly (0x27 or 0x3F on a real module, 0x20 / index 0 in Tinkercad).

Sensor conversion used in the sketch:

```
float voltage = analogRead(A0) * 5.0 / 1024.0;   // volts
float T = (voltage - 0.5) * 100.0;               // degrees C
```

Expected result: line 1 shows the temperature, line 2 shows the state message.



Base circuit running the provided starter sketch at 25 °C: line 1 shows the temperature (Temp: 24.7 C) and line 2 the state message (SAFE). Sensor input: the TMP36 on the breadboard, output to A0 (orange wire). Output: the I2C LCD at the bottom (SDA to A4, SCL to A5).

Part 2 files submitted in the ZIP: Circuit_Picture.png, Circuit_Design.brd, Schematic.pdf and the shared Tinkercad link: [Sizzling Waasa-Crift on Tinkercad]

Part 3: Testing

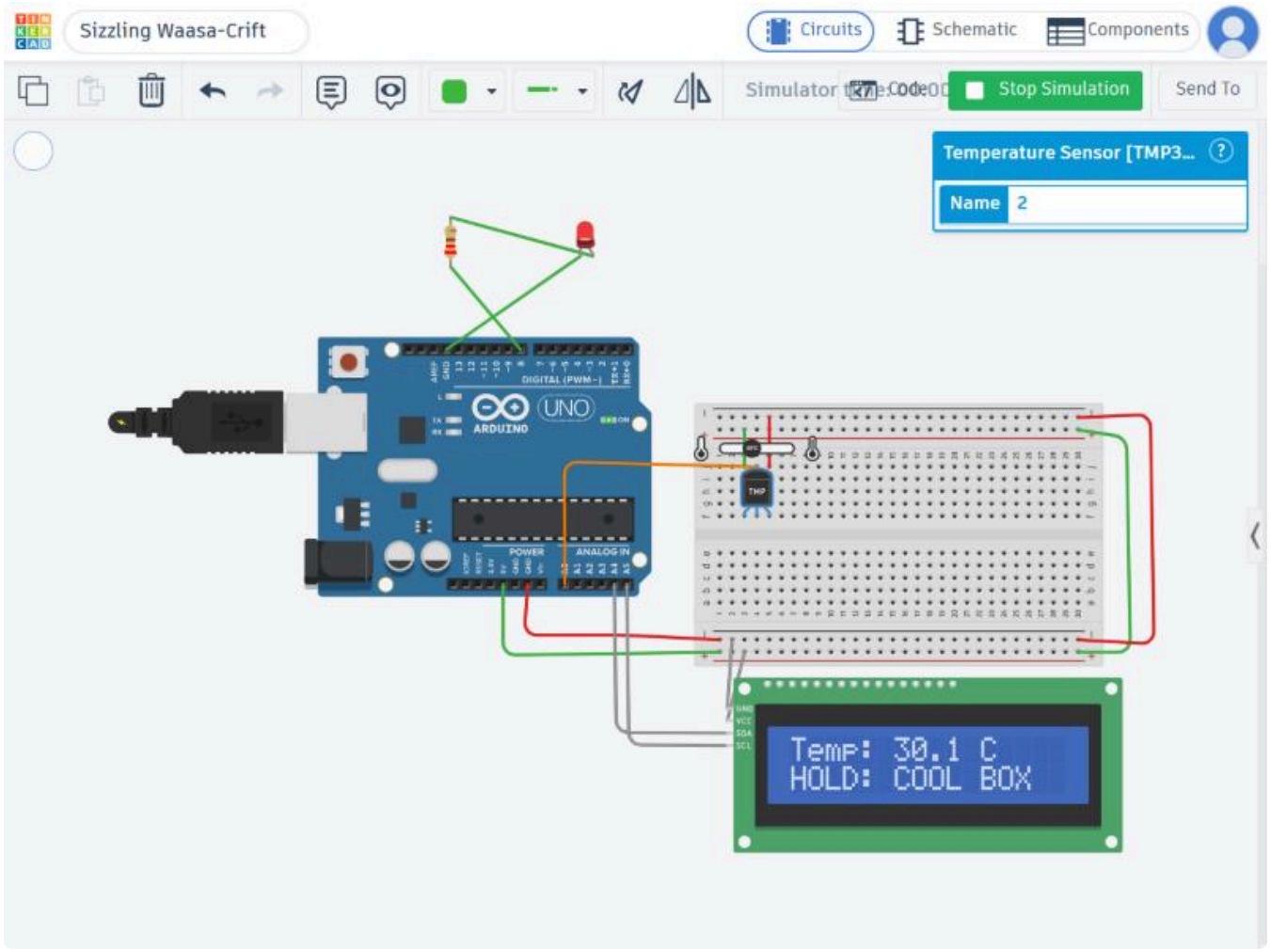
All 8 tests matched the FSM prediction. Tests were run in Tinkercad on 21 Oct 2026 with the final sketch, so line 2 shows the Part 4 messages; the temperature is the value shown on LCD line 1 (confirmed in the Serial Monitor).

Set temperature	Predicted LCD state	Observed LCD (line 1 / line 2)	LED	Match?
25 °C	SAFE	24.7 C / SAFE: OK	Off	Yes
28 °C	CAUTION	28.1 C / CHECK TEMP	Off	Yes
29 °C	CAUTION	29.1 C / CHECK TEMP	Off	Yes
30 °C	HOLD	30.1 C / HOLD: COOL BOX	On	Yes
31 °C	HOLD	31.1 C / HOLD: COOL BOX	On	Yes
32 °C	MOVE BOX	32.0 C / MOVE BOX NOW!	Blinking	Yes
33 °C	MOVE BOX	33.0 C / MOVE BOX NOW!	Blinking	Yes
Back to 25 °C	SAFE	25.7 C / SAFE: OK	Off	Yes

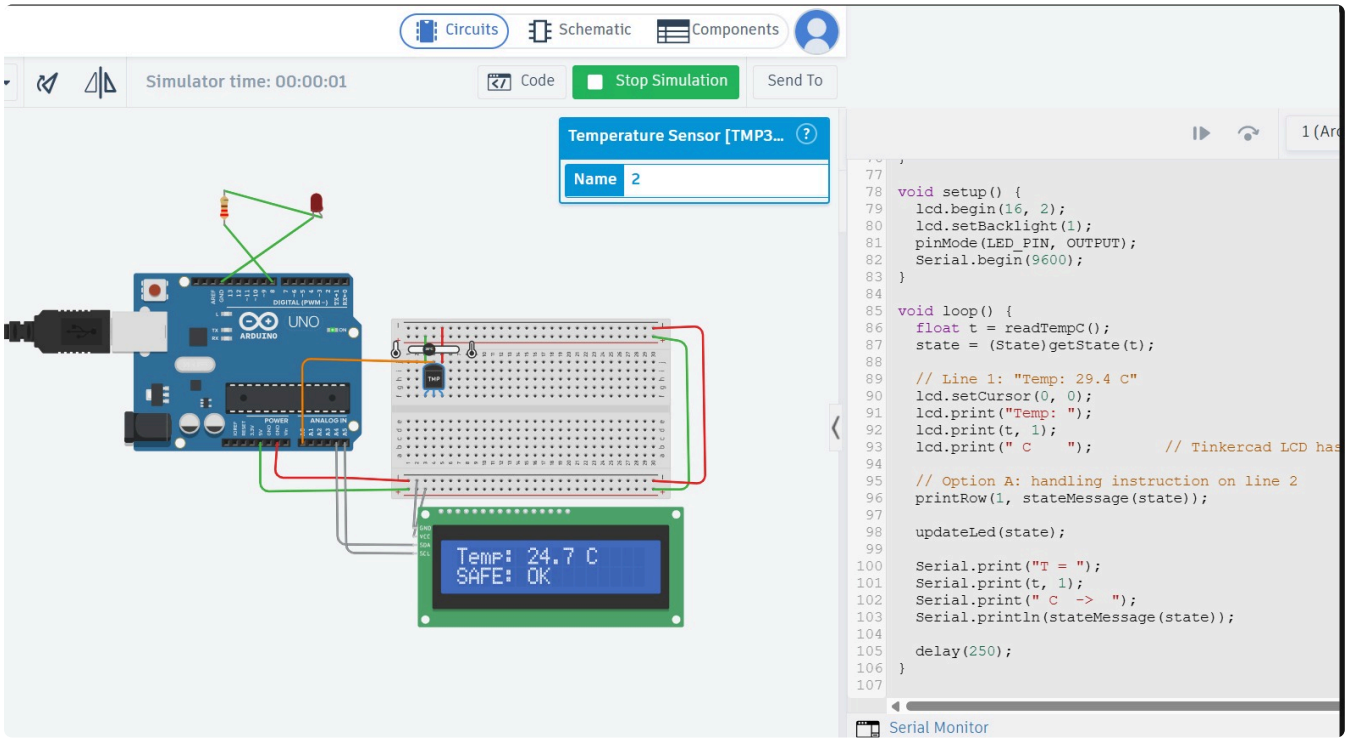
a) Jump from 25 °C to 33 °C: the LCD changes directly from SAFE: OK to MOVE BOX NOW!. The code checks the highest range first, so CAUTION and HOLD are skipped. In the test, the Serial Monitor went straight from 21.8 C SAFE to 33.0 C MOVE BOX with no CAUTION or HOLD line in between; cooling from 33 °C back to SAFE also skipped both middle states.

b) Why test exactly at 28, 30 and 32 °C: these are the boundaries, where errors usually hide. They show whether the code uses ⩾ or > correctly, so a box at exactly 30 °C is classed as HOLD, not CAUTION.

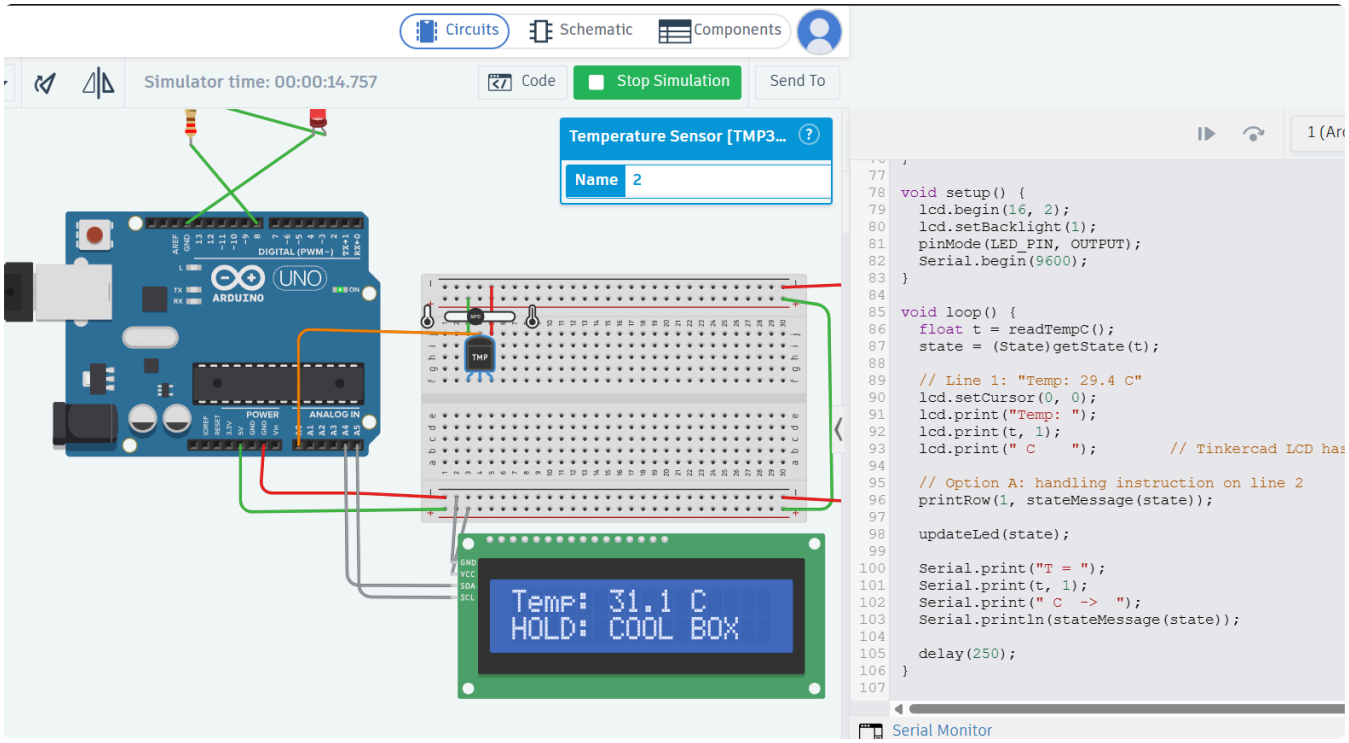
c) Differences from prediction: no state differed. The LCD temperature differed from the slider by up to 0.7 °C (24.7 C at 25 °C) because the ADC changes in steps of about 0.49 °C. We verified each result by reading the exact value in the Serial Monitor, and tested the thresholds 28, 30 and 32 °C while warming and again while cooling.



Threshold test: at 30 °C the LCD reads 30.1 C and switches to HOLD: COOL BOX; the LED turns on.



SAFE state at 25 °C: Temp: 24.7 C / SAFE: OK, LED off.



HOLD state at 31 °C: Temp: 31.1 C / HOLD: COOL BOX, LED on.

Part 4 – Engineering improvement (Option A)

The chosen improvement is Option A: line 2 of the LCD now gives a clear handling instruction instead of a short state label.

1. Problem addressed. The starter sketch showed short labels (SAFE, CAUTION, HOLD, MOVE BOX) that do not tell a carrier what to do, and it called `lcd.clear()` every 500 ms, which makes the screen flicker. The new display gives a clear instruction on line 2 and updates both lines in place without flicker.

2. Code changes (starting from the supplied sketch):

- Option A – the `switch` that printed the old labels is replaced by a function `stateMessage()` that returns a handling instruction for each state; each fits in 16 characters.
- Small display fix – `lcd.clear()` is removed; a helper `printRow()` writes each line and pads it with spaces, so the screen does not flicker and a short message never keeps leftover letters from a longer one. Line 1 keeps the starter format "Temp: xx.x C".
- The FSM logic moved into `getState()`, which checks the highest range first. It returns `int` instead of `State`, which fixed the Tinkercad compile error "'State' does not name a type".

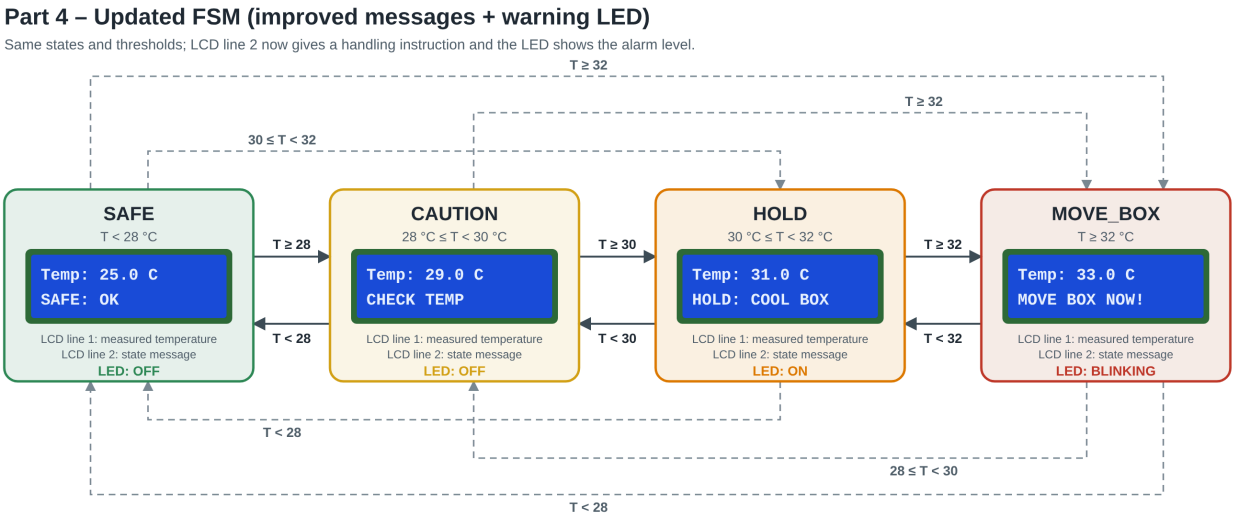
```
const char* stateMessage(int s) {
  switch (s) {
    case SAFE:     return "SAFE: OK";
    case CAUTION:  return "CHECK TEMP";
    case HOLD:     return "HOLD: COOL BOX";
    case MOVE_BOX: return "MOVE BOX NOW!";
  }
  return "";
}
```

State	Original line 2	New line 2	Characters
SAFE	SAFE	SAFE: OK	8
CAUTION	CAUTION	CHECK TEMP	10
HOLD	HOLD	HOLD: COOL BOX	14
MOVE_BOX	MOVE BOX	MOVE BOX NOW!	13

3. Tests of the improved prototype

Input	Starter LCD line 2	Improved LCD (line 1 / line 2)	Result
25 °C	SAFE	24.7 C / SAFE: OK	Clear instruction, no flicker
29 °C	CAUTION	29.1 C / CHECK TEMP	Tells the user what to do
33 °C	MOVE BOX	33.0 C / MOVE BOX NOW!	Urgent instruction
33 → 25 °C	SAFE	25.7 C / SAFE: OK	No leftover letters from the longer message

4. Updated FSM. Ranges and transitions are unchanged; only the outputs changed. LCD line 2 now shows the new handling messages, and each state also drives the bonus warning LED.



Updated FSM after the Part 4 improvement and the bonus LED.

Bonus – Warning LED

A red LED on pin 8 is OFF in SAFE and CAUTION, steady ON in HOLD, and blinks in MOVE_BOX, so the warning is visible even from a distance.

Wiring: pin 8 → 220 Ω resistor → LED anode (long leg); LED cathode (short leg) → GND. Current ≈ (5 − 2) V / 220 Ω ≈ 14 mA, below the Uno's 20 mA recommended pin current. Only a low-voltage LED is used, no high-current load.

Code changes: `pinMode(LED_PIN, OUTPUT)` in `setup()`, and a new function called every loop:

```
bool blinkOn = false;   // toggles every loop for the MOVE_BOX blink

void updateLed(int s) {
  blinkOn = !blinkOn;
  if (s == HOLD) {
    digitalWrite(LED_PIN, HIGH);                // steady ON
  } else if (s == MOVE_BOX) {
    digitalWrite(LED_PIN, blinkOn ? HIGH : LOW); // blink
  } else {
    digitalWrite(LED_PIN, LOW);                 // OFF
  }
}
```

The first version blinked with `(millis() / 250) % 2`. In the simulation each loop took longer than 250 ms, so the value always had the same parity and the LED stayed off in MOVE_BOX. A variable that flips every loop fixed it.

Input	State	Expected LED	Observed LED
25 °C	SAFE	Off	Off
29 °C	CAUTION	Off	Off
31 °C	HOLD	On (steady)	On
33 °C	MOVE_BOX	Blinking	Blinking (off and on in consecutive screenshots)

The full final sketch is submitted as `Arduino_Code_Final.ino` in the ZIP file (the Tinkercad version, tested in the simulation).

Part 5: Reflection

1. What did I contribute?

My part was building the base circuit in Tinkercad. I connected the TMP36 sensor, the I2C LCD and the Arduino, then loaded the starter code. I ran the simulation and changed the sensor temperature to check the messages on the LCD. I also took screenshots of the different states.

2. What did I learn?

I learned how a change in the sensor temperature changes the message on the screen. The Arduino reads the sensor voltage on A0, and the code calculates the temperature from that reading. It compares the result with 28, 30 and 32 °C to choose the state. For example, I set the sensor to 30 °C, and the screen showed 30.1 C. The message changed from CAUTION to HOLD because the calculated temperature had reached 30 °C.

3. What problem did I encounter?

One thing I noticed was that the temperature on the LCD did not exactly match the sensor setting. When I selected 25 °C, it showed 24.7 C. I think this is because the Arduino reads the voltage in small steps before the code converts it into a temperature. It still showed SAFE at that setting. I would test more values near the limits to check whether the small difference changes the state earlier or later than expected.

4. What would I improve next?

I would make a better box for the circuit. I would leave an opening where the LCD can be read easily and enough room inside to keep the wires from getting squeezed. This would help protect the circuit when moving the box.